

1 Pile = 3 bananas

↓ 2 Piles = 6 bananas

same.

Q9

arr = [3 6 7 11]

Koko ~~must~~ has to eat all of these bananas in 8 hrs.

So,



3hr

~~1+1+1~~

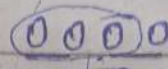
X eating speed
1b/hr



6hr

~~1+1+1+1+1+1~~

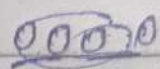
2



9hr

~~1+1+1+1+1+1+1+1+1~~

3



12hr

~~1+1+1+1+1+1+1+1+1+1+1+1~~

4

total = 3+6+7+11

= 27 > 8 hrs.

X eating speed.
2b/hr.

total = 2+3+4+6

= 15 > 8 hrs.

X eating speed.
3b/hr.

total hrs = 1+2+3+4

= 10 > 8 hrs.

✓ eating speed
4b/hr.

total hrs = 1+2+2+3

= 8 < 8 hrs

✓ eating speed.
5b/hr.

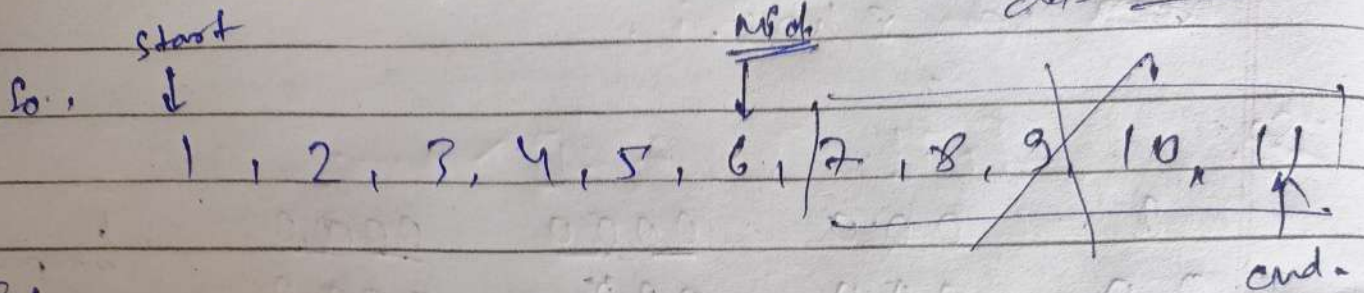
total, hr = 8 hrs

→ This is the minimum speed of eating banana.

And the maxi. speed of eating banana is 11b/hr. by this Koko eat all bananas in 4 hrs.

min speed = 1

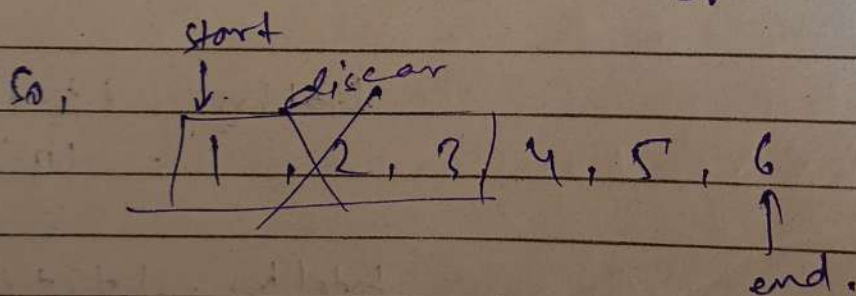
max speed = 11.



Bisect

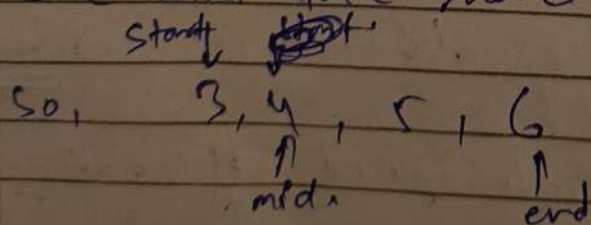
$$\text{mid} = 1 + (11 - 1) / 2$$
$$= 1 + \frac{10}{2} = 6.$$

i.e. Koko eat 6 banana, per hrs and our given hrs is 8. So Koko has a chance to slow down her speed.



$$\text{mid} = 1 + (6 - 1) / 2 = 3.$$

Koko has eat 3 in a hrs, so for all bananas she will take more than 8 hrs.



$$\text{Bisect} = 3 + (6 - 3) / 2 = 4.$$

Koko has eat 4 in a hrs, so for all bananas she will take 8 hrs which is the minimum required times to eat.

As move forward.

When $(start = end)$, this is ~~point~~ optimum value ~~at~~ within $K0K0$ ~~at~~ eat all bananas in given specific time.

```
int start = 0 / ;  
int end = 0 ;  
for (int pile : piles) {  
    end = Math.max(end, pilesstart);  
}
```

```
while (start < end) {  
    int mid = start + (end - start) / 2 ;  
    int hrs = 0 ;  
    for (int pile : piles) {  
        hrs += (pile + mid - 1) / mid ;  
    }  
    if (hrs <= h) {  
        end = mid ;  
    } else {  
        start = mid + 1 ;  
    }  
}  
return start ;  
}
```